MULTI-AGENT PATH FINDING · PROJECT REVIEW

AisleFlow

Forty years of multi-agent path finding, the warehouse problem it grew into, and what a directional, congestion-aware lifelong planner actually bought us.

5
PLANNERS COMPARED

183
TESTS IN THE SUITE

0
RUNTIME DEPENDENCIES

LDA-PIBT — Lifelong Aisle-Managed Priority Inheritance with Backtracking. AisleFlow is the implementation; LDA-PIBT is the algorithm.

WHAT THIS DECK COVERS

Three questions

1. **Where did MAPF come from?**
   From decoupled prioritised planning in 1987, through optimal solvers, to the greedy planners that made thousands of robots tractable.

2. **What problem were we solving?**
   Warehouse aisles are narrow and effectively one-way. PIBT is direction-blind. LDA-PIBT adds an aisle layer that decides direction, and a recovery layer that survives it.

3. **How, and did it work?**
   A two-layer architecture, six repaired mechanisms, and a hypothesis suite that scores each claim on the quantity it actually names. Three of six hold.

The design principle that runs through all three: direction, congestion and turning cost decide **which legal move PIBT tries first** — they never replace its collision checks or its backtracking.

THE PROBLEM

Multi-agent path finding

Given a graph and k agents, each with a start and a goal, find k paths such that every agent arrives and no two agents ever collide.

Time is discrete. At each timestep an agent moves along one edge or waits. Everything else in MAPF — makespan, sum-of-costs, or the throughput objective this project uses — is an optimisation layered on top of that one rule.

Two kinds of collision define the problem. They are the next slide, and they are the **only** two things the low level of this system checks.

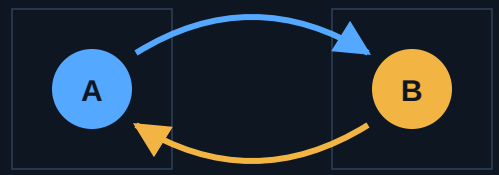
THE PROBLEM

Two collisions, and nothing else

both claim the shaded cell at t+1

Vertex conflict

Two agents occupy the same cell at the same timestep.



legal for each agent alone, illegal for the pair

Swap conflict

Two agents exchange cells across one edge in one timestep.

The joint action space is exponential in the agents

Each agent may move in four directions or wait. For k agents planned together, the joint branching factor is 5^k.

Coupled search is therefore exponential in the number of agents, not in the size of the map. Doubling the floor space is cheap; adding ten robots is not.

$\approx 5^k$

JOINT BRANCHING FACTOR

Optimal MAPF is NP-hard

NP-hard for both makespan and sum-of-costs (Yu & LaValle, 2013), anticipated by Ratner & Warmuth on the generalised 15-puzzle in 1986.

So optimality is not a matter of finding a cleverer algorithm. Any planner that scales is giving something up, and the interesting question is which thing.

NP-hard

TO SOLVE OPTIMALLY

In a warehouse, the horizon never ends

A delivered task is immediately replaced by a new one. There is no final goal configuration to plan to.

The classical formulation does not get harder here. It stops applying. This is the turn that made lifelong MAPD a separate field, and it is the setting this project lives in.

Endless

NO TERMINAL STATE

So the field split. One branch chased optimality and paid in scale. One gave up optimality for completeness. A third gave up both for speed — and that turned out to be the branch warehouses needed.

HISTORY · ERA 1 OF 5

Decoupled: plan one agent at a time

Erdmann & Lozano-Pérez, 1987

Plan each agent in turn, treating the already-planned ones as moving obstacles.

Silver's Cooperative A* / WHCA*, 2005

Adds a shared reservation table over space-time, so later agents can see where earlier ones will be.

Fast, **incomplete**, and sensitive to the planning order — the ordering problem that priority inheritance later attacks directly.

HISTORY · ERA 2 OF 5

Optimal: search the conflicts, not the joint space

CBS — Sharon et al.

Find a conflict, split on it, and re-plan the two single agents under the resulting constraints. Optimal and genuinely elegant.

ICTS, M*, branch-and-cut-and-price

Different decompositions of the same idea, all following within a few years.

And it falls over past tens of agents in dense maps: the conflict tree grows with the conflicts, and a dense warehouse is nothing but conflicts.

HISTORY · ERA 3 OF 5

Rule-based: complete by construction

Push and Swap, Push and Rotate, TASS

A fixed repertoire of local manoeuvres that provably terminates in polynomial time.

The trade

Solution quality is poor — but they never fail to terminate, which no decoupled planner can promise.

This era is where the seven-level recovery ladder later in this deck comes from: when a general planner is stuck, fall back on rules that always finish.

HISTORY · ERA 4 OF 5

Lifelong: the warehouse changes the question

Kiva / Amazon Robotics, 2008

Hundreds of robots, narrow aisles, tasks that never stop arriving. Wurman, D'Andrea & Mountz.

MAPD and Token Passing, 2017

Ma, Li, Kumar & Koenig formalise pickup-and-delivery as a continuous stream.

RHCR, 2021

Li et al. replan a rolling window, resolving collisions only within the horizon.

Token Passing and RHCR are the two baselines this project measures itself against.

HISTORY · ERA 5 OF 5

Scalable: one timestep at a time

PIBT — Okumura, Machida, Défago & Tamura

IJCAI 2019; *Artificial Intelligence*, 2022. Priority inheritance and backtracking, resolved within a single timestep. This is the algorithm this repository extends.

LaCAM, 2023

Builds a complete search on top of PIBT, recovering guarantees without giving up its speed.

Thousands of agents, in real time.

HOW THE FAMILIES TRADE OFF

You choose two of three: optimal, complete, fast

FAMILY	REPRESENTATIVES	GUARANTEE	SCALE	THE PRICE
Optimal search	CBS, ICTS, M*, BCP	Optimal + complete	Tens of agents	Exponential in conflicts
Bounded suboptimal	ECBS, EECBS	Within a factor of optimal	Low hundreds	Focal search overhead
Rule-based	Push and Swap, Push and Rotate	Complete, polynomial	Hundreds	Poor solution quality
Greedy / reactive	PIBT, LaCAM, this repo	Conditional progress only	Thousands, real time	No global optimality

AisleFlow lives on the bottom row. The question the project asks: can a warehouse-aware layer on top of a greedy planner **buy back** some of the quality the greedy choice gave away?

THE TURN THAT MADE THIS PROJECT NECESSARY

From one-shot MAPF to lifelong MAPD

Classical MAPF

k agents, k goals, fixed in advance.

Plan once, offline, then execute.

Success is every agent parked on its goal.

Objective: makespan or sum-of-costs.

The plan is valid until it finishes.

Lifelong MAPD

Tasks arrive as a stream — Poisson, bursty or scheduled.

A robot that delivers is immediately re-assigned.

There is no terminal configuration to plan to.

Objective: throughput, service time, and its tail.

Every plan is provisional; you replan forever.

THE CORE WE BUILD ON

PIBT, in four steps

1. **Candidates**
   The four neighbours of the current cell, plus staying put. Nothing longer than one step is ever planned.

2. **Hard rejection**
   Off-graph, kinematics, vertex conflict, swap conflict. Nothing else — a rule that deletes a legal move breaks the progress argument.

3. **Score and sort**
   Survivors are ranked by `S(v)`. Direction, congestion, turning cost and aisle rules all act here, and only here.

4. **Inherit, then backtrack**
   If the best cell holds an unplanned robot, that robot plans next with the caller's priority. If it fails, undo and try the next candidate.

Cost per timestep `O(|A|(Δ(G) + F + log|A|))`. Measured at **1.00 ms/step** for 40 robots on `warehouse_medium`; 1.90 with the aisle layer on.

THE CORE WE BUILD ON

Priority inheritance, in one picture

R1 moves aside, so R3 moves, so R7 moves. If nobody can, the chain unwinds and R7 tries its second-best cell.

THE GAP WE SET OUT TO CLOSE · 1 OF 2

PIBT does not know what an aisle is

single-file corridor — no passing

each is moving optimally, right up to the moment neither can move at all

Direction-blind

PIBT scores a cell by how much closer it gets you. Nothing in that score knows the corridor has a flow.

No structural memory

Nothing records that this corridor was flowing north five steps ago, so the flow flaps with the traffic instead of settling.

THE GAP WE SET OUT TO CLOSE · 2 OF 2

And the matching is blind to it too

Task assignment picks whichever robot is nearest. But nearest *through a jammed corridor* is not nearest — and a greedy match keeps feeding robots into the jam that made it slow.

Two blind spots, one cause: no part of the system has a representation of the corridor as a shared resource with a state.

> Robots generate directional requests, but aisles make directional decisions.
>
> the architectural commitment of LDA-PIBT, stated verbatim in the code

A robot may **prefer** a direction; the aisle **owns** it — and the aisle is the only thing that can see every robot that wants to use it.

HOW IT IS BUILT

Two layers, and a line neither crosses

HIGH LEVEL — WHAT EACH ROBOT WANTS

task assignment · waypoints · routes · directional demand · aisle direction · hysteresis + drain · reservations · priorities

deadlock detection and recovery

ranks candidates and prices the ones it dislikes — never removes them

constrains + ranks candidates

LOW LEVEL — PIBT, ONE SYNCHRONISED TIMESTEP

candidates · hard rejection · score-sorted · priority inheritance · backtracking · validate every step

the only place collision-freedom is decided

Zero vertex and zero swap conflicts across every map, density, seed and variant tested — checked every timestep by `validate_plan`.

HOW, PART 1

One scoring function carries the whole high level

$$S_i(v) = \alpha \cdot \text{progress} + \beta_i \cdot [\text{preferred direction}] + \gamma_i \cdot [\text{stays in aisle}] - \lambda \cdot \text{turn} - \mu \cdot \text{congestion} - \nu \cdot \text{wait} - \xi \cdot [\text{bottleneck}] - \zeta \cdot [\text{counterflow}]$$

α = 10.0 · progress

Steps closer to the waypoint. Dominates everything else by design.

ζ = 8.0 · counterflow

The price of driving against a committed direction. This is where the aisle layer acts.

λ = 0.5 · turning

A flat penalty per direction change. The cheapest term here, and the strongest single mechanism in the system.

HOW, PART 1

The ordering is the design: α > ζ > β > γ > λ

A whole step of progress beats everything. Driving the wrong way down an aisle beats every other preference. A single turn is the cheapest thing you can be asked to pay.

ζ < α, and ζ is a price

The aisle terms are penalties, not rejections. A robot drives the wrong way when that is the only way to make progress — and pays for it.

μ·C must be normalised

Unnormalised, the congestion mixture is a robot *count* and μ·C reaches the scale of α·Δ — measured mean 3.40, p90 5.75, max 9.60. Congestion then outranks the terms it is meant to modulate.

Both conditions were false in the code until this review. The ordering the score claimed was not the ordering it had.

HOW, PART 2

Aisles decide, and hold the decision

REVERSE is the exact mirror of FORWARD, with the sign of the imbalance flipped

HOW, PART 2

Two clocks, not one

Demand

Each robot contributes weighted demand for FORWARD or REVERSE in the aisle it is about to enter. The aisle sums both sides.

Minimum green

A committed direction is locked for T_min steps, and flips only outside a ±τ dead band — so it cannot flap with the traffic.

Maximum green

Past T_max, any opposing demand forces a flip. Without it a balanced aisle starves the side that never wins the imbalance.

Entry reservations

A robot at an entrance gets a ticket only if occupancy plus pending tickets stay under capacity. Tickets expire after 15 steps, so a stalled robot cannot hold a corridor hostage. Direction restricts *entry*, not interior movement — and it is priced, not enforced.

HOW, PART 3

A stall is only a stall when something corroborates it

In dense lifelong traffic a robot waits `t_blocked` steps as a matter of course. "No progress" is therefore a *precondition*, never a trigger.

The precondition

A group of robots has made no route progress for several timesteps.

The corroboration

A wait-for cycle in the dependency graph, or a repeated configuration in the position history. Only then does recovery escalate.

Worth **0.022 → 0.134 tasks/step** on `warehouse_corridors`. Levels 5–7 are destructive when nothing is actually stuck.

HOW, PART 3

Seven levels, one per timestep

1. **Recompute routes from scratch**
2. **Unlock and re-decide the affected aisles**
3. **Release stale aisle reservations**
4. **Escalate the blocked robots' priorities**
5. **Allow temporary reverse movement**
6. **Send robots to escape vertices**
7. **Drop all directional constraints locally**

One level per timestep, not all seven at once: in a synchronous loop you cannot observe whether a remedy worked until the next step. Firing all seven would apply six of them **blind**.

WHAT WE FOUND WHEN WE CHECKED · DEFECT 1 OF 4

The aisle layer broke our own invariant

Aisle direction and reservations were implemented as **hard rejections**, deleting legal moves before scoring — while the architecture slide says the high level never replaces PIBT's backtracking.

Priority inheritance needs to be able to push a robot aside. A rejected move is a move it can no longer use to unblock a chain.

1.9–3.1×

THROUGHPUT, PRICING INSTEAD OF REJECTING

Measured on `corridors` 0.084 → 0.158, `narrow` 0.112 → 0.304, `medium` 0.161 → 0.405 tasks/step; **p = 0.008** on each.

WHAT WE FOUND WHEN WE CHECKED · DEFECT 2 OF 4

Two hypotheses were never actually tested

H3's treatment cell was empty

The reservation layer was gated on `direction_control == "aisle"`, and the variant built to isolate it sets `"robot"`. The treatment was a bit-identical copy of the control, so its main effect was necessarily zero.

And on one map it never fires

On `bottleneck`, no aisle ever commits a direction at all — so the mechanism under test was never active.

"Exactly zero effect" described **the wiring**, not the mechanism. A result that clean should have been suspicious on its own.

WHAT WE FOUND WHEN WE CHECKED · DEFECT 3 OF 4

The aisle signal could starve

A committed direction was held indefinitely unless the imbalance crossed τ. But pickups on one side and deliveries on the other keeps demand permanently *balanced*.

So the imbalance never crossed the threshold, the direction never flipped, and the traffic wanting the other way never moved.

35 / 35

ROBOTS STALLED BY T = 120, CORRIDORS

Fixed by the maximum green: past `T_max`, any opposing demand at all forces a drain and a flip.

WHAT WE FOUND WHEN WE CHECKED · DEFECTS 4–6

Three more, each measured

Bent aisles

Aisles were being segmented into L and U shapes, so one-waying a corner blocked it in *both* directions.

Recovery on healthy traffic

Escalation fired on ordinary queueing, applying destructive remedies to a system that was working.

Congestion outranked its own targets

The unnormalised mixture let μ·C dominate the terms it was meant to modulate.

---

183 tests, up from 161. Task assignment is held identical across all five planners, so the baseline comparison isolates the movement layer and nothing else.

RESULT 1 · AGAINST THE LITERATURE

Plain PIBT wins, and not narrowly

`lifelong_pibt` beats both external baselines on every map, **p < 0.001** on throughput in every cell. Token Passing completes **literally zero** tasks on `warehouse_corridors` across all ten seeds.

Why: no inheritance

All 16 robots queue nose-to-tail in the connecting corridor and never move again. A queue Token Passing forms, it cannot resolve.

And it is 56–356× cheaper

```
lifelong_pibt   1.01 ms/step
full_lda_pibt   4.06
RHCR   85.4
token_passing   318.4
```

10 seeds × 400 timesteps, permutation test vs `lifelong_pibt`. Assignment is held constant, so neither baseline runs its own paper's policy and RHCR's window is an untuned default.

RESULT 1 · WHERE LDA-PIBT LANDS

The aisle layer wins where corridors are the constraint

bottleneck

`full_lda_pibt` now wins: 0.14 vs 0.13, **p = 0.016**.

corridors

Ties, where it used to lose 0.02 to 0.16.

medium and narrow

Still loses to plain PIBT. Turning cost alone beats the full aisle layer on both.

The honest summary: the aisle layer earns its cost exactly where the map makes direction a real constraint, and costs more than it returns everywhere else.

RESULT 2 · DE-CONFOUNDING

With both cells live, the attribution reverses

On `warehouse_medium`, once H3's treatment cell actually differed from its control:

Aisle direction alone

`0.337 → 0.405`

Entry admission alone

`0.337 → 0.262`

Both together

`0.217`

Two mechanisms that were reported as one. Separated, one helps and one hurts — and together they are worse than either.

RESULT 2 · DE-CONFOUNDING

One flag, three mechanisms

`congestion_aware` moved the movement score, the assignment cost and the blocking term *together*.

So H4 — "congestion in matching cuts service time" — was never a test of congestion in matching. It was a test of all three at once, and the same confound the factorial designs were built to remove, one level further down.

The general lesson: an ablation flag that controls more than one mechanism is not an ablation. It is a **bundle**, and its effect size cannot be attributed to anything.

RESULT 3 · THE SCOREBOARD

Each claim, scored on its own metric

	CLAIM	VERDICT
H1	Aisle direction lifts per-aisle flow in narrow aisles	**not on its own metric**
H2	Hysteresis reduces switching and prevents oscillation	**supported, every map**
H3	Entry capacity admission cuts head-on conflicts	**map-dependent**
H4	Congestion in matching cuts service time	**not supported**
H5	Proximity decay of the direction weight helps arrival	**mechanism is inert**
H6	Localised recovery beats a global planner	**supported on corridors**

Scored on the quantity each claim **names** — head-on conflicts for H3, per-aisle flow for H1 — not on throughput. H1 fails its own metric yet raises end-to-end throughput on three of four maps.

WHAT THE FIXES WERE WORTH

Three mechanisms, measured one at a time

1. **Direction ranks candidates instead of rejecting them**
   The largest effect in the repository: 0.084 → 0.158 on corridors, 0.112 → 0.304 on narrow, 0.161 → 0.405 on medium, p = 0.008 on each.

2. **Recovery escalates only on a corroborated stall**
   0.022 → 0.134 tasks/step on `warehouse_corridors` — invisible before, because the hard constraints were manufacturing the deadlocks recovery was rescuing.

3. **The maximum green, and why it now matters less**
   Decisive while direction was a hard constraint. Once counterflow is merely expensive a robot can buy its way out, so the two fixes are partly redundant. It stays because it makes the layer correct, not just survivable.

Coordinated direction assignment across parallel aisles — the last review's most promising fix — is implemented, and measures **±0.011**.

WHERE THIS GOES

Next, in order of expected payoff

1. **Make entry admission earn its cost**
   The one repaired mechanism that still loses throughput on every map. Both the capacity model and the admission rule are suspect.

2. **Work out why the aisle layer loses on open grids**
   It wins on corridors and loses on medium and narrow. Turning cost alone beats it on two maps.

3. **Tune the baselines honestly, then re-run**
   Pair each baseline with the assignment policy from its own paper, and size RHCR's window for its intended scale.

4. **Close the gaps the spec still leaves open**
   Intersection reservations, kinematics and footprints, asynchronous execution, and the delayed/failed-robot scenarios.

IN CLOSING

Three things worth taking away

1. **The core was the contribution all along**
   Forty years of MAPF converged on a mechanism — priority inheritance with backtracking — that a hundred lines of recursion capture. Plain lifelong PIBT still wins on the open grids.

2. **A mechanism that cannot act is not a negative result**
   Three of six hypotheses were reported as failures. One had an empty treatment cell, one a map where the treatment never fired, one was rescuing deadlocks we had manufactured.

3. **The invariant was load-bearing, not decorative**
   "Ranks candidates, never decides safety" appears three times in this deck. The code did the opposite, and that one line accounts for every collapse we blamed on aisle management.

`python3 main.py — no install, no dependencies, 183 tests, zero collisions.`